Manual Didático do WSL

Guia em português para estudantes de Sistemas Operacionais

Figura de abertura – Boas-vindas Windows para Linux.

Este material foi estruturado para uso em aula, estudo individual e apoio à resolução de exercícios, nem todas as imagens representam a implementação completa dos algoritmos. As cores no software WSL podem variar.

Sumário

1. Conceitos e definições 4
   1. Evolução do WSL 4
   2. Arquitetura e Funcionamento 4
   3. Integração com o Sistema Windows 4
   4. Gerenciamento de Distribuições 5
   5. Sistema de Arquivos 5
   6. Redes e Processos 5
   7. Uso em Ambientes Acadêmicos e Profissionais 5
   8. Vantagens e Limitações 6
   9. Considerações Finais 6
2. Instalação 7
   Passo a passo de instalação do WSL2 no Windows 7
   Passo 1: 7
   Passo 2: 7
   Passo 3: 7
   Passo 4: 7
   Passo 5: 8
   Passo 6: 8
   Passo 7: 9
   Passo 8: 9
3. Compilando um código em C: 10
   1. Baixando o código de exemplo: 10
   2. Compilando o exemplo: 10
   3. Executando o programa com sucesso 11
4. Executando seu próprio código 11
   1. Criando seu primeiro arquivo: 11
   2. Iniciando o código em C 13
      O que esse código faz? 13
   3. Um exemplo de programa que imprime uma linha na tela 13
      O que estamos fazendo? 13
   4. Compilando seu primeiro código: 14
      Compilar usando o VSCode Terminal: 14
      Compilar usando o VSCode Terminal: 16
   5. Organizando o código e adicionando comentários. 17

Adicionando comentários ao nosso código de exemplo: 17
Apêndice A - Glossário rápido 18
Apêndice B – Código de exemplo comentado. 20

1. Conceitos e definições

O **Windows Subsystem for Linux (WSL)** é uma camada de compatibilidade desenvolvida pela Microsoft que permite a execução de distribuições Linux diretamente no ambiente do Windows, sem a necessidade de virtualização tradicional ou de dual boot. Seu principal objetivo é oferecer um ambiente Linux funcional e integrado, facilitando o desenvolvimento, os testes e o ensino de conceitos de sistemas operacionais.

1. Evolução do WSL

O WSL possui duas versões principais, com diferenças arquiteturais relevantes:

- **WSL 1**: Implementa uma camada de tradução de chamadas de sistema (syscalls), convertendo chamadas Linux em chamadas equivalentes do kernel Windows (NT Kernel). Não há kernel Linux real.
- **WSL 2**: Introduz um kernel Linux completo, executado em uma máquina virtual leve (utilizando tecnologia de virtualização baseada em Hyper-V). Proporciona maior compatibilidade e desempenho, especialmente em operações de I/O e no uso de containers.

2. Arquitetura e Funcionamento

No WSL 2, a arquitetura é composta por:

- **Kernel Linux real** mantido pela Microsoft
- **Máquina virtual leve (lightweight VM)** otimizada para inicialização rápida
- **Sistema de arquivos virtualizado**, com integração entre Windows (/mnt/c) e Linux (/home, /etc, etc.)
- **Comunicação eficiente entre host e guest**, com baixo overhead

Essa abordagem permite executar binários ELF nativamente, garantindo compatibilidade com ferramentas e aplicações Linux.

3. Integração com o Sistema Windows

O WSL oferece alto nível de integração entre os dois sistemas operacionais:

- Acesso ao sistema de arquivos do Windows via /mnt/
- Execução de comandos Linux no terminal Windows (PowerShell, CMD ou Windows Terminal)
- Execução de aplicações Windows a partir do ambiente Linux (interop)
- Compartilhamento de rede e recursos de hardware

Essa integração cria um ambiente híbrido, útil tanto para desenvolvimento quanto para aprendizado.

4. Gerenciamento de Distribuições

O WSL permite instalar múltiplas distribuições Linux, como:

- Ubuntu
- Debian
- Kali Linux
- openSUSE

Cada distribuição é isolada, permitindo diferentes ambientes de configuração no mesmo sistema.

Comandos principais:

```
wsl --list --verbose     # Lista distribuições instaladas
wsl --install           # Instala o WSL e uma distro padrão
wsl --set-version <distro> 2   # Define uso do WSL 2
```

5. Sistema de Arquivos

O WSL utiliza dois contextos de sistema de arquivos:

- **Sistema Linux (ext4 virtualizado)**: Melhor desempenho, localizado internamente na VM
- **Sistema Windows (NTFS montado)**: Acessado via /mnt/c, /mnt/d, etc.

Boas práticas:

- Trabalhar com arquivos do projeto dentro do sistema Linux para melhor performance
- Evitar operações intensivas diretamente no sistema montado do Windows

6. Redes e Processos

- O WSL 2 possui uma interface de rede virtual (NAT)
- O Linux e o Windows podem se comunicar via localhost
- Processos Linux são gerenciados dentro do kernel Linux da VM
- Suporte a ferramentas como SSH, servidores web e containers (Docker)

7. Uso em Ambientes Acadêmicos e Profissionais

O WSL é amplamente utilizado para:

- Ensino de Sistemas Operacionais (processos, arquivos, permissões)
- Desenvolvimento em linguagens como C, Python, Node.js
- Execução de ferramentas de linha de comando (gcc, make, bash)
- Ambientes DevOps e containers (Docker, Kubernetes)

8. Vantagens e Limitações

Vantagens:

- Integração nativa com Windows
- Baixo consumo de recursos comparado a VMs tradicionais
- Alta compatibilidade (especialmente no WSL 2)
- Facilidade de instalação e uso

Limitações:

- Dependência do Windows como sistema host
- Algumas diferenças em comportamento de hardware e drivers
- Desempenho variável em acesso a arquivos externos (/mnt)

9. Considerações Finais

O WSL representa uma evolução significativa na convergência entre sistemas operacionais, permitindo que usuários utilizem ferramentas Linux em um ambiente Windows sem complexidade adicional. Para fins educacionais, ele se torna uma ferramenta estratégica, pois elimina barreiras de acesso e possibilita experimentação prática com conceitos fundamentais de sistemas operacionais.

2. Instalação

Esse é um passo para instalar o WSL no Windows 11 com o PowerShell. Para mais detalhes, siga a tutorial disponível em: https://learn.microsoft.com/pt-br/windows/wsl/install

O objetivo dessa instalação é permitir o uso de Linux no Windows para compilar código em C.

Passo a passo de instalação do WSL2 no Windows

Siga os passos a seguir para instalar o Linux Ubuntu 24.04 em ambiente WSL2 para Windows:

Passo 1:

Abrir o Windows PowerShell.

Passo 2:

Execute o comando a seguir para instalar o Ubuntu-24.04 :

```
wsl.exe --install Ubuntu-24.04
```

O resultado do comando deve ser:

```
PS C:\Users\PICHAU> wsl.exe --install Ubuntu-24.04
Baixando: Ubuntu 24.04 LTS
```

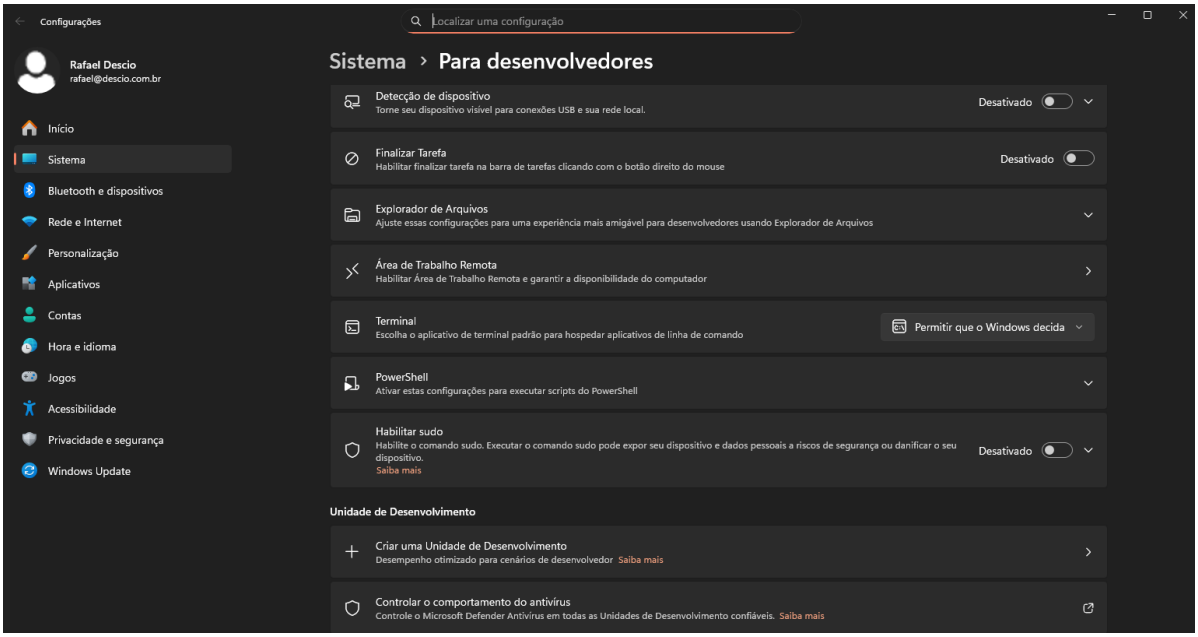
```
PS C:\Users\PICHAU> wsl.exe --install Ubuntu-24.04
Baixando: Ubuntu 24.04 LTS
Instalando: Ubuntu 24.04 LTS
Distribuição instalada com êxito. Ele pode ser iniciado por meio de 'wsl.exe -d Ubuntu-24.04'
Iniciando Ubuntu-24.04...
Provisioning the new WSL instance Ubuntu-24.04
This might take a while...
Create a default Unix user account: descio
New password:
Retype new password:
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU$  sudo apt update
[sudo] password for descio:
0% [Connecting to archive.ubuntu.com] [Connecting to security.ubuntu.com]
```

Passo 3:

Reinicie seu computador.

Passo 4:

Habilite o sudo, vá nas Configurações do Windows -> Sistemas -> Para Desenvolvedores. Localize a configuração “Habilitar sudo” e habilite:



Passo 5:

Abra novamente o Windows PowerShell.

Atualize o WSL para a versão 2 com o comando:

```
wsl.exe -set-default-version 2
```

A saída do comando deve ser:

```
PS C:\Users\PICHAU> wsl.exe --set-default-version 2
Para informações sobre as principais diferenças com o WSL 2, visite https://aka.ms/wsl2
```

Configure o Ubuntu-24-04 como padrão na versão WSl 2:

```
wsl.exe --set-version Ubuntu-24-04 2
wsl.exe --set-default Ubuntu-24.04 2
```

A saída do comando deve ser:

```
PS C:\Users\PICHAU> wsl.exe --set-default Ubuntu-24.04 2
A operação foi concluída com êxito.
```

Passo 6:

Ainda dentro do PowerShell, entre no modo wsl:

```
wsl -shutdown
wsl
```

```
PS C:\Users\PICHAU> wsl --shutdown
PS C:\Users\PICHAU> wsl


descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU$
```

***** Veja que, quando estamos no modo WSL o terminal aparece em cores diferentes, da mesma forma que seria apresentado no Linux.***

Atualize os pacotes do Linux Ubuntu com o comando:

```
sudo apt update
```

Passo 7:

Instale o pacote build-essential.

```
sudo apt install build-essential
```

O resultado do comando deve ser:

```
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU$  sudo apt install build-essential
[sudo] password for descio:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

Quando solicitada a autorização para instalar, selecione 'Sim' (Y).

Passo 8:

Verifique a instalação do pacote build essential com o comando:

```
gcc -- version
```

O resultado deve ser:

```
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

3. Compilando um código em C:

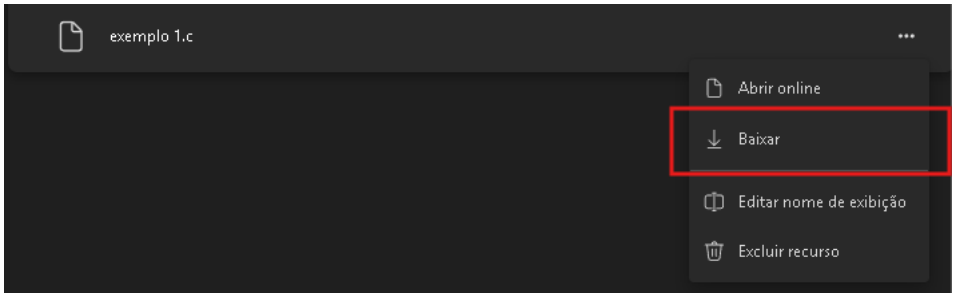
Para nosso tutorial utilizamos o código de exemplo disponibilizado em classe.

1. Baixando o código de exemplo:

Acesse o Teams, na equipe de Sistema Operacionais.

Localize o arquivo “exemplo 1.c” nas Atividade de Sala.

Baixe o arquivo.



Em nosso exemplo, ele foi baixado para o diretório “Downloads”.

Acesse o diretório Downloads no ambiente WSL2:

Utilize o comando:

```
cd <caminho>
```

No caso desse exemplo:

```
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU$  cd Downloads/
```

Liste os arquivos para encontrar o arquivo baixado, isso pode ser feito usando o comando:

```
ls
```

A saída do comando deve listar:

```
'exemplo 1.c'
```

2. Compilando o exemplo:

Agora, iremos compilar o arquivo em C para gerar o executável.

Garanta que você esteja na pasta onde o arquivo “exemplo 1.c” está.

```
gcc exemplo\ 1.c -o programa_exemplo
```

Quando a execução ocorre sem erros, esse comando não gera saída:

```
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Downloads$  gcc exemplo\ 1.c -o programa_exemplo
```

Caso todos os passos tenham sido seguidos corretamente, agora temos um arquivo executável chamado “programa_exemplo”.

Para executar no WSL2 utilize o comando:

```
./programa_exemplo
```

O resultado do comando deve ser:

```
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Downloads$ ./programa_exemplo
Erro ao abrir arquivo
```

Nosso programa de exemplo realiza a leitura de um arquivo chamado “dados.txt”. Quando esse arquivo não existe na pasta, a saída do programa exibe “Erro ao abrir arquivo”

3. Executando o programa com sucesso

Agora, realizaremos a criação e a leitura do arquivo “dados.txt”.

Há diversas maneiras de criar um arquivo TXT; caso prefira, utilize o Bloco de Notas ou outro editor de texto de sua preferência. Aqui segue uma maneira simplificada de criar no Linux.

Execute o comando a seguir para criar o arquivo “dados.txt” com o conteúdo “Teste de SO”:

```
echo “Teste de SO” > dados.txt
```

O resultado deve ser a criação do arquivo dados.txt com o conteúdo especificado.

```
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Downloads$ echo "Teste de SO" > dados.txt
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Downloads$
```

Execute novamente o executável “programa_exemplo” e veja a saída:

```
./programa_exemplo
```

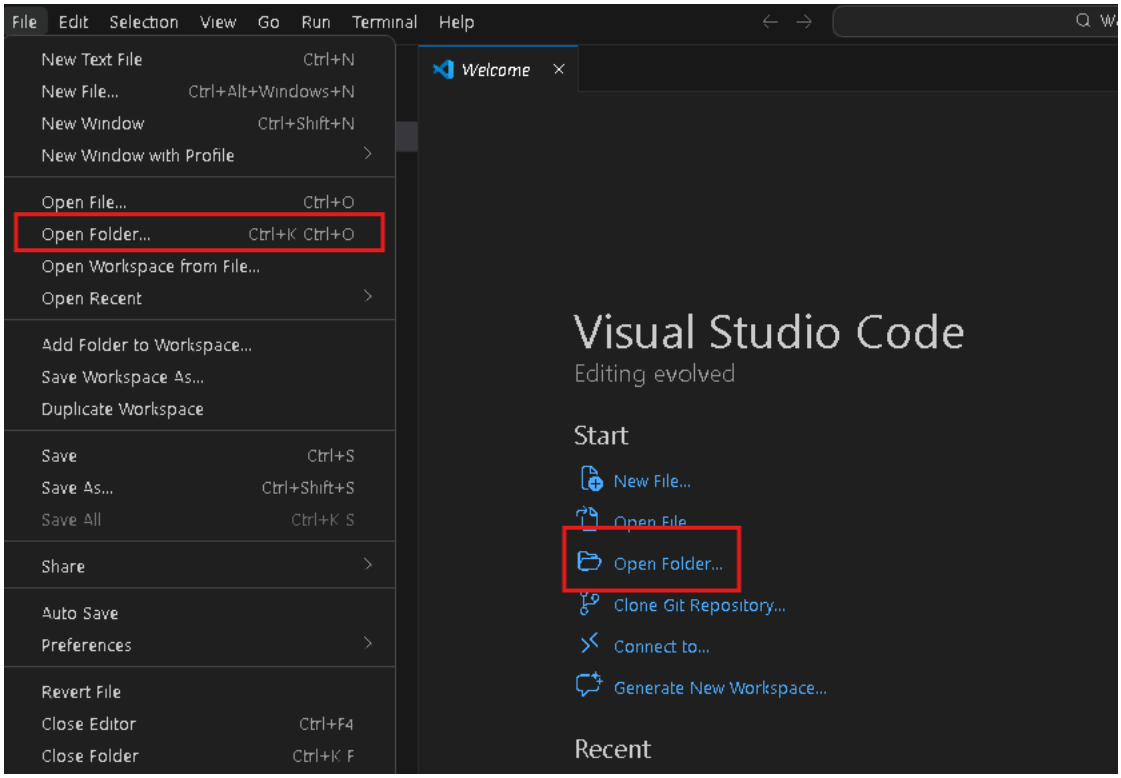
4. Executando seu próprio código

Para criar seu próprio código em C recomenda-se utilizar uma IDE (Integrated Development Environment ou, em português, Ambiente de Desenvolvimento Integrado). Em nosso exemplo utilizaremos o Microsoft VSCode.

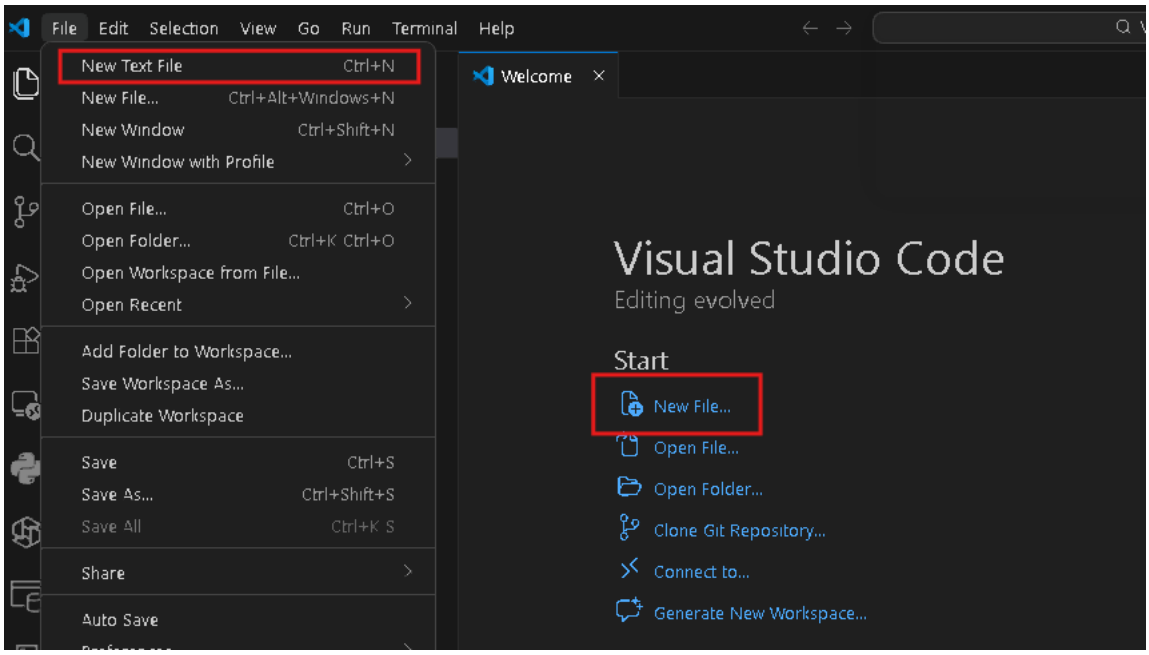
Para manter a organização de seus arquivos, recomenda-se criar uma pasta para cada projeto que você for executar.

1. Criando seu primeiro arquivo:

Após criar a pasta onde seus códigos serão armazenados, abra o VSCode e selecione “Abrir diretório” ou “Open Folder” no menu.

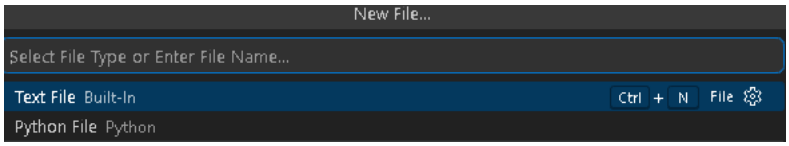


Agora crie novo arquivo de texto através do menu “Arquivo” ou selecionando CTRL+N:



O resultado será a criação de um arquivo sem título e sem extensão:

Selecione Arquivo de Texto:



2. Iniciando o código em C

Para criar nosso primeiro arquivo em C, utilizaremos como base o seguinte código:

```
#include <stdio.h>
int main() {}
```

Esse código servirá de base para o programa que iremos desenvolver.

O que esse código faz?

A linha 1 importa a biblioteca stdio.h, que utilizaremos para fazer a leitura e escrita de arquivos do sistema e demais comunicação com dispositivos de entrada/saída.

```
#include <stdio.h>
```

A linha 2 inicia uma função chamada “main”. Essa função é obrigatória em todos os programas em C. É por meio dela que o sistema sabe por onde começar a executar o seu código.

```
int main() {}
```

Mais informações podem ser encontradas no link: https://learn.microsoft.com/pt-br/cpp/c-language/main-function-and-program-execution?view=msvc-170

3. Um exemplo de programa que imprime uma linha na tela.

Agora que já temos a base do nosso programa, vamos imprimir uma linha na tela. Para isso, vamos alterar nosso código, escrevendo dentro da função “main()”, como o exemplo a seguir:

```
#include <stdio.h>

int main() {
    printf("Minha primeira linha");
}
```

O que estamos fazendo?

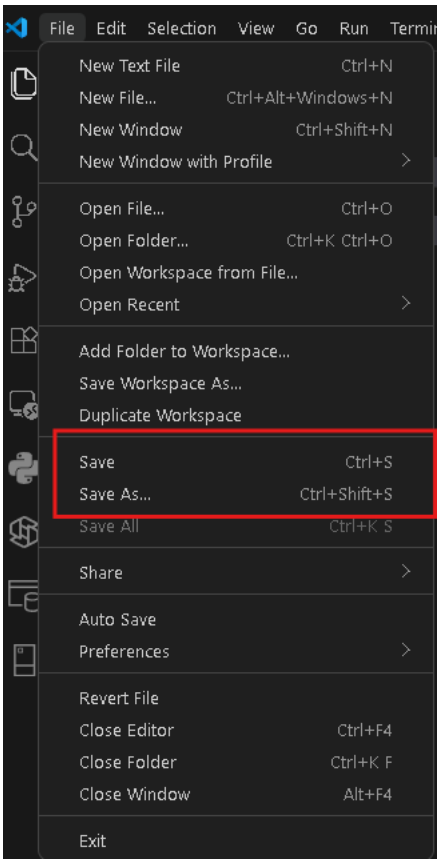
Adicionamos a linha:

```
    printf("Minha primeira linha");
```

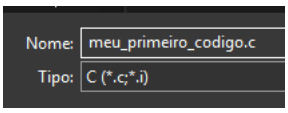
Dentro da função main().

4. Compilando seu primeiro código:

Salve o arquivo fonte com a extensão ".c", no VSCode vá até a opção Salvar (Save):



Digite o nome que deseja salvar; em nosso exemplo, utilizaremos "meu_primeiro_codigo.c":

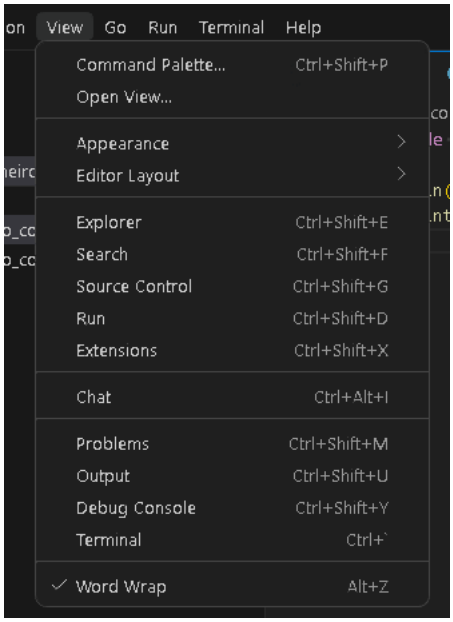


Com o arquivo salvo, já podemos compilar nosso código. Para isso utilizaremos o comando "gcc", que instalamos na Seção Instalação.

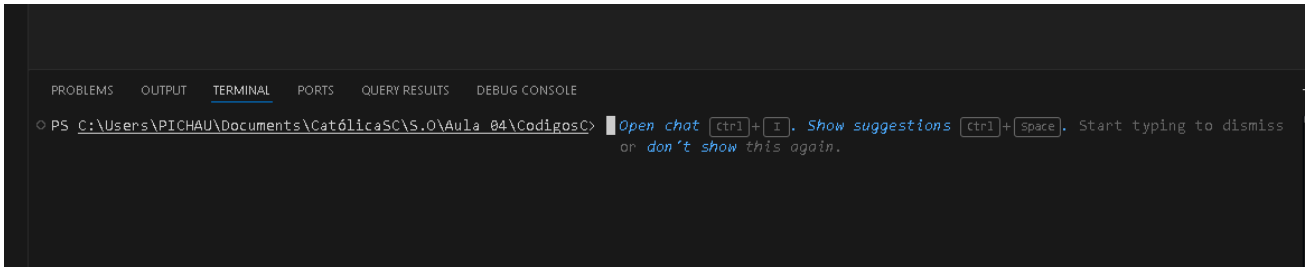
Há diversas maneiras de acessar o terminal e executar o comando *gcc*; faremos isso de duas formas.
A primeira através do Terminal do VSCode, a segunda através do PowerShell.

Compilar usando o VSCode Terminal:

Acesse o menu Exibir e selecione Terminal:

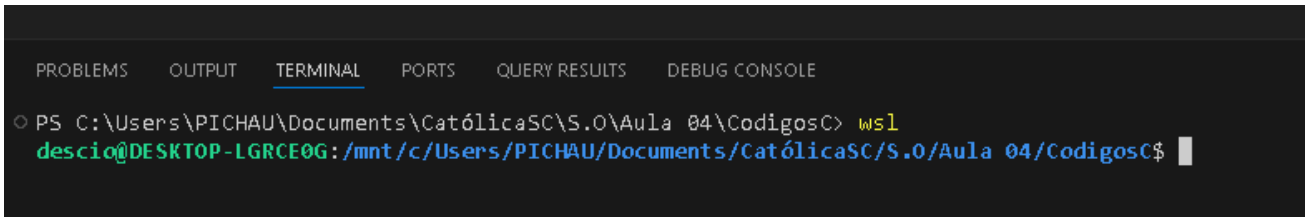


A seguir, deve ser apresentado um terminal na sua interface do VSCode, como no exemplo:



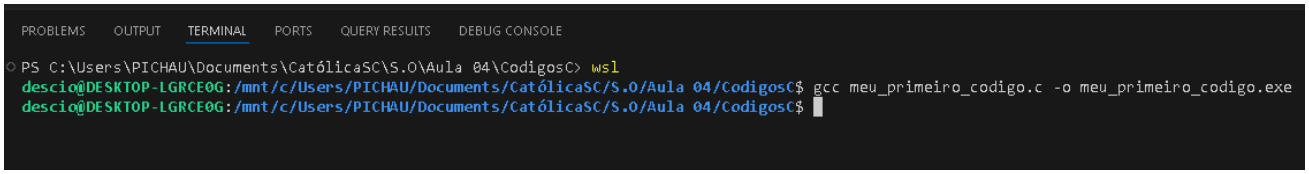
Habilite o modo WSL2 utilizando o comando:

```
wsl
```



Compile seu programa utilizando o GCC utilizando o comando:

```
gcc meu_primeiro_codigo.c -o meu_primeiro_codigo.exe
```



Execute o programa criado.

```
PROBLEMS   OUTPUT   TERMINAL   PORTS   QUERY RESULTS   DEBUG CONSOLE

PS C:\Users\PICHAU\Documents\CatólicaSC\S.O\Aula 04\CodigosC> wsl
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Documents/CatólicaSC/S.O/Aula 04/CodigosC$ gcc meu_primeiro_codigo.c -o meu_primeiro_codigo.exe
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Documents/CatólicaSC/S.O/Aula 04/CodigosC$ ./meu_primeiro_codigo.exe
Minha primeira linhadescio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Documents/CatólicaSC/S.O/Aula 04/CodigosC$
```

A saída do terminal deve exibir o texto “Minha primeira linha”.

Compilar usando o VSCode Terminal:

Abra o Windows PowerShell.

Navegue até a pasta que está seu código.

```
Windows PowerShell

O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS C:\Users\PICHAU>  cd '.\Documents\CatólicaSC\S.O\Aula 04\CodigosC\'
PS C:\Users\PICHAU\Documents\CatólicaSC\S.O\Aula 04\CodigosC>
```

Habilite o modo WSL2 com o comando:

```
wsl
```

```
descio@DESKTOP-LGRCE0G: /

O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS C:\Users\PICHAU>  cd '.\Documents\CatólicaSC\S.O\Aula 04\CodigosC\'
PS C:\Users\PICHAU\Documents\CatólicaSC\S.O\Aula 04\CodigosC> wsl
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Documents/CatólicaSC/S.O/Aula 04/CodigosC$
```

Execute o comando GCC para compilar seu código:

```
gcc meu_primeiro_codigo.c -o meu_primeiro_codigo.exe
```

```
descio@DESKTOP-LGRCE0G: /

O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS C:\Users\PICHAU>  cd '.\Documents\CatólicaSC\S.O\Aula 04\CodigosC\'
PS C:\Users\PICHAU\Documents\CatólicaSC\S.O\Aula 04\CodigosC> wsl
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Documents/CatólicaSC/S.O/Aula 04/CodigosC$ gcc meu_primeiro_codigo.c -o meu_p
rimeiro_codigo.exe
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Documents/CatólicaSC/S.O/Aula 04/CodigosC$
```

Execute o programa criado.

```
descio@DESKTOP-LGRCE0G: /
O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS C:\Users\PICHAU>  cd '.\Documents\CatólicaSC\S.O\Aula 04\CodigosC\'
PS C:\Users\PICHAU\Documents\CatólicaSC\S.O\Aula 04\CodigosC> wsl
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Documents/CatólicaSC/S.O/Aula 04/CodigosC$ gcc meu_primeiro_codigo.c -o meu_p
rimeiro_codigo.exe
descio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Documents/CatólicaSC/S.O/Aula 04/CodigosC$ ./meu_primeiro_codigo.exe
Minha primeira linhadescio@DESKTOP-LGRCE0G:/mnt/c/Users/PICHAU/Documents/CatólicaSC/S.O/Aula 04/CodigosC$
```

5. Organizando o código e adicionando comentários.

Na linguagem C, há duas maneiras de adicionar comentários ao código. Os comentários ajudam a documentar e organizar melhor seu código, eles são textos ignorados pelo compilador.

Adicionando comentários ao nosso código de exemplo:

```c
#include <stdio.h>

int main() {
    // Esse é um comentário
    printf("Minha primeira linha"); // Esse é outro comentário
    /* Esse também é um comentário em
    múltiplas linhas*/
}
```

A primeira e mais simples maneira de adicionar um comentário é utilizar barras duplas (//). Todo texto após as barras será ignorado pelo compilador. Esse tipo de comentário é usado para documentar uma única linha. Em nosso exemplo temos:

```c
    // Esse é um comentário
```

A segunda maneira é utilizar blocos de comentários. Esse formato de comentários é utilizado com a barra asterisco (/*) e a barra asterisco (*/). Essa é uma maneira de documentar várias linhas, na qual todo conteúdo entre o início (/*) e o fim (*/) será ignorado pelo compilador. No nosso exemplo temos:

```c
/* Esse também é um comentário em
    múltiplas linhas*/
```

Apêndice A - Glossário rápido

WSL (Windows Subsystem for Linux)

Camada que permite executar um ambiente Linux dentro do Windows, sem necessidade de máquina virtual tradicional.

Distribuição Linux (Distro)

Sistema operacional baseado em Linux instalado no WSL (ex: Ubuntu), que fornece terminal, bibliotecas e ferramentas de desenvolvimento.

Terminal (Shell)

Interface de linha de comando para executar instruções no sistema. No WSL, normalmente utiliza-se o Bash.

Bash

Interpretador de comandos padrão em muitas distribuições Linux, responsável por executar comandos digitados no terminal.

Diretório

Equivalente a uma pasta. É utilizado para organizar arquivos no sistema de arquivos Linux.

Caminho (Path)

Endereço de um arquivo ou diretório no sistema. Exemplo: /home/user/projeto.

Arquivo fonte (.c)

Arquivo de texto contendo código escrito na linguagem C.

Compilar

Processo de transformar código-fonte (ex: .c) em um programa executável. Envolve análise, tradução e geração de código de máquina por meio de um compilador.

Compilador (gcc)

Ferramenta que realiza a compilação. O gcc (GNU Compiler Collection) é o compilador mais utilizado para C no Linux.

Executável

Arquivo gerado após a compilação, que pode ser executado pelo sistema operacional. No Linux, geralmente não possui extensão (ex: ./programa).

Comando gcc

Instrução utilizada para compilar código C. Exemplo:

```
gcc programa.c -o programa
```

Parâmetro -o

Define o nome do arquivo executável gerado pelo compilador.

Execução (./)

Forma de executar um programa no diretório atual. Exemplo:

```
./programa
```

Permissões de arquivo

Regras que definem quem pode ler, escrever ou executar um arquivo. No Linux, é necessário permissão de execução para rodar programas.

Pacote build-essential

Conjunto de ferramentas essenciais para compilação no Linux (inclui gcc, make, etc.).

Gerenciador de pacotes (apt)

Ferramenta usada para instalar softwares no Linux. Exemplo:

```
sudo apt install build-essential
```

sudo

Comando que permite executar ações com privilégios administrativos.

Apêndice B – Código de exemplo comentado.

```
#include <stdio.h>

int main() { //função principal do C, utilizada para iniciar um programa.

    // Solicita ao Sistema Operacional a leitura de um arquivo chamado "dados.txt" e
salva na memória da variável "f"
    FILE *f = fopen("dados.txt", "r");

    // Verifica se a variável f tem valor nulo. Caso positivo, escreve "Erro ao
abrir arquivo" e sai do programa.
    if (f == NULL) {
        printf("Erro ao abrir arquivo\n");
        return 1;
    }

    // Declara as variáveis que serão utilizadas para armazenar em memória os
conteúdos das linhas e caracteres.
    int linhas = 0;
    int caracteres = 0;
    int c;

    // Criar um laço de repetição (loop), para ler os caracteres do arquivo
"dados.txt".
    while ((c = fgetc(f)) != EOF) {
        // Soma +1 na variável caracteres.
        caracteres++;

        // Quando encontra um caractere de final de linha "\n", soma +1 ao número de
linhas.
        if (c == '\n') {
            linhas++;
        }

        // Escreve na tela quando está lendo os caracteres da linha "2"
        if (linhas == 1 && c != '\n') {
            // estamos lendo a linha 2
            printf("%c", c);
        }
    }

    // Soma +1 ao final, para contar a última linha.
    if (caracteres > 0) {
        linhas++;   // conta a última linha
```

```
    }

    // Escreve na tela o total de linhas e caracteres.
    printf("\nTotal de linhas: %d\n", linhas);
    printf("Total de caracteres: %d\n", caracteres);

    // Fecha o arquivo "dados.txt", para liberar recursos do sistema operacional.
    fclose(f);

    // Retorna zero para o Sistema Opercional, indicando que o código foi executado
com "Sucesso".
    return 0;
}
```